

BUILD Worksheet — 45 Minutes

Block 5 · Write a reference doc for your domain

You're about to write a reference doc for one domain in the workshop warehouse. This sheet is your clock and your prompts. The actual document you're filling in is `references/domain-doc-template.md` — open both side by side.

The rule for today: run the query before you write the line. Every fabricated number in this pack's own

history got caught only by someone executing something and reading the result. You're joining that habit now, not

being warned about it.

Before you start — set up your query tool

You'll run database queries using a small Python file. **You don't need to know Python or SQL** — you'll copy pre-written queries, paste them in, and run them.

Open `explore.py` in the main `analytics-skills` folder (it's already in the repo). You'll see a short script that looks like this:

```
import duckdb

con = duckdb.connect("warehouse/people_analytics.duckdb", read_only=True)

result = con.execute("""
    SELECT * FROM fct_separation LIMIT 5
""").fetchall()

for row in result:
    print(row)

con.close()
```

The only part you'll change is the SQL between the `"""` marks (the triple quotes). Everything else stays the same.

To run it, open your terminal (not VS Code's play button — use the terminal you ran `check_setup.py` from earlier) and type:

Windows: `python explore.py` **Mac/Linux:** `python3 explore.py`

You should see rows of data. To try a different query, change only the text between the `"""` marks, save, and run again.

If you get "Cannot open file... being used by another process": close any DuckDB viewer in VS Code. If you clicked on `people_analytics.duckdb` in the file explorer, or installed a DuckDB extension, it holds a lock on the

file. Close that tab, then try running your query again.

If you're not comfortable with SQL: use the **Starter Queries** handout (printed, or open `workshop/starter-queries.html` from the repo). It has pre-written queries for your domain — copy them in, run them, and follow the "Try changing..." prompts.

A 30-second SQL primer — just enough for today:

- SELECT = which columns to show
 - FROM = which table to look in
 - WHERE = a filter (only show rows matching this condition)
 - GROUP BY = summarize: instead of one row per person, show one row per category
 - ORDER BY = sort the results
 - count(*) = "how many rows"
 - avg(...) = "what's the average"
 - LIMIT 5 = "just show me the first 5 rows"
-

Before you start — save your reference doc

Copy the template to a file named after your domain:

Domain	Save as
Attrition	<code>references/attrition.md</code>
Compensation	<code>references/compensation.md</code>
Engagement	<code>references/engagement.md</code>

Windows: `copy references\domain-doc-template.md references\attrition.md`

Mac/Linux: `cp references/domain-doc-template.md references/attrition.md`

(Replace `attrition` with your domain name.)

Edit your copy, not the template. The ablation script in the last block looks for a file with your domain's exact name — if it's called anything else, it won't find it.

0. Orient — 2 min

Pull up your **Define worksheet** (the question spec) and your **Validate worksheet** (the assertions and prediction). You picked your domain back in Define — this block is where you find out what your question spec got right and wrong.

Domain	Start here
Attrition	<code>fct_separation</code> , <code>rpt_attrition_monthly</code>
Compensation	<code>fct_compensation</code>
Engagement	<code>fct_engagement_survey</code>

If you're pairing up, confirm you're both still working the same domain you claimed this morning.

1. Quick Reference — 5 min

Answer these before touching SQL:

- What does one row of your canonical table represent?
- What's the person key? (Hint: it might not be `worker_id` — check `EXAMPLE-headcount.md`'s Gotcha 2 for why.)
- What's the as-of convention — a snapshot, or an activity window over a period?

Write the **Quick Reference** section now, even if it's your best guess. You'll correct it in step 3.

2. Required Filters — 5 min

Run your first query — open your `explore.py`, paste the starter query for your domain (from the Starter Queries handout, or write your own), and run it. Look at the results. Now ask:

- Does this table mix worker types, statuses, or currencies that your question shouldn't blend together?
- What's the smallest change to your filter that meaningfully changes the result? (Try adding or removing a condition after `WHERE` in the query.)

Write the **Required Filters** table with at least two rows.

3. Gotchas — 15 min the point of the exercise

This is the section a model cannot write for you, because it requires actually looking. Budget the most time here.

Before you query: glance at your Validate worksheet's prediction. You guessed whether a plain agent would pass or fail your assertions. Keep that guess in mind — you're about to find out whether the trap you anticipated is the trap that's actually here.

The move: find two plausible ways to answer the same simple question from your domain, and see if they agree.

- **Attrition** — `rpt_attrition_monthly` gives you a numerator. It does not give you a denominator. Compute the rate three defensible ways using headcount from `dim_worker_snapshot` (beginning of period? end?

average?) and see how far apart the three answers land.

- **Compensation** — pull every row's currency. Now compute a naive average across all of them. Does that number mean anything?
- **Engagement** — pull one survey item across every wave in the table. Before you trend it, check whether every wave measured it the same way.

When you find a discrepancy: **that's your Gotcha**. Write it up using this format in your reference doc:

- **What:** one sentence describing the trap
- **Why:** why this happens (the data, not a mistake)
- **Do:** the correct approach (with a query if you have one)
- **Don't:** the wrong approach that looks right (with the query that produces the wrong number)

See `references/EXAMPLE-headcount.md` for a worked example of this format.

Aim for at least **one** fully-written Gotcha with real numbers from your own query. One real one beats three guessed ones.

Stuck after 8 minutes? Flag the facilitator. Don't spend the whole block searching — say what you've tried and ask for a nudge.

4. Measures — 5 min

Name the one measure your domain is really about (an attrition rate, an average salary, an engagement score). Write down:

- The exact numerator and denominator
 - What population is in scope
 - Whether you found more than one definition already in use in the data
-

5. Query Patterns — 10 min

Write one query that answers the single most common question someone would ask about your domain — and **run it**. It goes in the doc only if it executed successfully against the warehouse.

If you have time left, write a second pattern.

6. Wrap — 5 min

Fill in whatever's left, in this order of priority:

1. **Cross-References** — one row: what domain would someone join yours to, and what would go wrong if they weren't careful?
2. **Escalate, Don't Guess** — one line: a question in your domain you should route to a human rather than answer.
3. **Open Questions** — anything you didn't get to. An honest gap beats a confident guess.

Then run the checklist at the bottom of `domain-doc-template.md`. If you can't check every box, that's fine — say which one is unchecked when we reconvene.

Share-out — when the facilitator calls time

Be ready to say, in one breath:

| "My domain was ___. My Gotcha is ___. Without it, someone would have gotten ___ instead of the right answer."

That's it. If your Gotcha is "I didn't find one yet," say that too — a documented near-miss is still useful, and it's an honest place to start the correction-harvesting loop after today.

If you finish early

Open someone else's domain and try to break their Query Pattern with a question they didn't anticipate. That's a free adversarial-review rep, and it's exactly what `sql-reviewer` will formalize in the next block.